

Eje D — mixed precision, `torch.compile` y medicion de rendimiento

Fecha de implementacion: 2026-07-19

Este documento recoge el analisis tecnico, las decisiones de implementacion, los cambios de codigo, las validaciones realizadas y los protocolos de benchmarking definidos durante la sesion de trabajo del Eje D.

El objetivo del Eje D es mejorar el aprovechamiento del hardware disponible en CompressARC sin modificar la arquitectura del modelo ni la perdida MDL. En concreto, se implementaron mecanismos configurables para:

- usar precision mixta (`bf16` / `fp16`) en entrenamiento secuencial y paralelo;
- ajustar la precision de matmul de PyTorch;
- activar `torch.compile` de forma experimental;
- medir si estas opciones compensan realmente en `parallel_train.py`;
- comparar runs antes/despues con metricas especificas del Eje D.

La implementacion se diseno para ser reversible: por defecto el comportamiento sigue siendo FP32 sin compilacion, y las mejoras se activan solo mediante flags.

1. Contexto tecnico

CompressARC entrena un modelo independiente por tarea ARC. En entrenamiento paralelo, `parallel_train.py` lanza un proceso por tarea mediante `multiprocessing`. Cada worker ejecuta `solve_task.solve_task`, que crea:

1. un `Task` de `preprocessing.py`;
2. un `ARCCompressor` nuevo;
3. un optimizador Adam;
4. un `solution_selection.Logger`;
5. un loop de entrenamiento de `n_train_iterations` llamadas a `train.take_step`.

La ruta caliente queda asi:

```
parallel_train.py
    parallelize_runs(...)
        multiprocessing.Process(target=solve_task.solve_task, ...)
            solve_task.py
                model = arc_compressor.ARCCompressor(task)
                for train_step in range(n_train_iterations):
                    train.take_step(task, model, optimizer, train_step, logger, ...)
                    model.forward()
                        layers.decode_latents(...)
                        share_up -> softmax -> cummax -> shift -> direction_share
                        -> nonlinear -> share_down -> normalize      x4
                    loss = total_KL + 10 * reconstruction_error
                    backward + optimizer.step
```

El Eje H ya habia reducido parte del cuello de botella de CPU mediante:

- cache de VRAM de Phase 1;
- `--max-workers`;
- `--postprocess-stride`;
- materializacion tardia de curvas en `Logger.materialize_curves()`;
- vectorizacion de crops en `train.take_step`.

El Eje D parte de ese estado y busca aprovechar mejor GPU/matrix cores, especialmente en AMD RDNA4 / ROCm, donde BF16 es la ruta mas interesante.

2. Uso del servidor Codebase Memory MCP

Antes de leer o modificar archivos se uso el servidor Codebase Memory MCP, siguiendo la regla del repositorio.

Proyecto detectado y usado:

```
home-asier-repos-arc-agi-src-SuperCompressARC
```

La consulta de arquitectura mostro clusters relevantes:

- `arc_logging` / `parallel_train`: donde vive `run_split` y `parallelize_runs`;
- `runtime_config`: nuevo cluster tras la implementacion, conectado con `solve_task`, `take_step`, `ARCCompressor` y `Logger`;
- `layers`: donde vive el nucleo del forward pass;

- `profile_parallel_train`: donde se extendieron las metricas.

La memoria de codigo confirmo que el cambio debia atravesar cuatro superficies:

1. configuracion comun de runtime;
2. paso de entrenamiento secuencial;
3. worker paralelo;
4. scheduler/profiler paralelo.

3. Hipotesis tecnica del Eje D

La hipotesis local fue:

BF16 y la precision de matmul pueden reducir coste de computo/memoria en el hot path sin alterar la semantica del modelo, siempre que los terminos MDL criticos, especialmente KL y loss, se acumulen en FP32.

Para `torch.compile`, la hipotesis fue mas cauta:

`torch.compile` solo compensa si el coste inicial de compilacion por tarea se amortiza durante las iteraciones de Phase 2. Como `parallel_train.py` crea un proceso y un modelo por tarea, el coste de compilacion se paga muchas veces, no una sola vez.

La condicion de break-even es:

```
tiempo_compile_por_tarea < ahorro_por_iteracion * n_iteraciones
```

Con `n_iteraciones = 1500`, si compilar una tarea tarda 120 s, el ahorro minimo necesario es:

```
120 s / 1500 = 0.080 s/iter = 80 ms/iter
```

Por eso `torch.compile` se implemento como bandera experimental y no como valor por defecto.

4. Archivos modificados

4.1 `runtime_config.py` nuevo

Se anadio un modulo nuevo para centralizar la configuracion del Eje D.

Responsabilidades:

```
MIXED_PRECISION_CHOICES = ('off', 'bf16', 'fp16')
COMPILE_FORWARD_CHOICES = ('off', 'default', 'reduce-overhead', 'max-autotune')
MATMUL_PRECISION_CHOICES = ('highest', 'high', 'medium')
```

`apply_torch_backend_settings`

Aplica ajustes compartidos por runners secuenciales y paralelos:

```
def apply_torch_backend_settings(float32_matmul_precision='high'):
    if float32_matmul_precision:
        torch.set_float32_matmul_precision(float32_matmul_precision)
    if hasattr(torch.backends, 'cudnn'):
        torch.backends.cudnn.benchmark = True
    cuda_backend = getattr(torch.backends, 'cuda', None)
    cuda_matmul = getattr(cuda_backend, 'matmul', None)
    if cuda_matmul is not None and hasattr(cuda_matmul, 'allow_tf32'):
        cuda_matmul.allow_tf32 = True
```

Notas:

- `torch.set_float32_matmul_precision('high')` permite a PyTorch elegir rutas de matmul mas rapidas cuando el backend lo soporta.
- `cudnn.benchmark` se mantiene por compatibilidad y no rompe si el backend existe.
- `allow_tf32` solo tiene efecto real en GPUs NVIDIA con TF32; en AMD/ROCm puede ser no-op, pero se comprueba defensivamente.

`autocast_context`

Encapsula precision mixta:

```
def autocast_context(mixed_precision):
    if mixed_precision == 'off':
        return contextlib.nullcontext()
    dtype = torch.bfloat16 if mixed_precision == 'bf16' else torch.float16
    return torch.autocast(device_type='cuda', dtype=dtype)
```

Decisiones:

- `off` conserva exactamente el comportamiento FP32.
- `bf16` es la opcion recomendada en RDNA4 porque mantiene el rango numerico de FP32.
- `fp16` queda disponible para experimentos, pero es mas arriesgado numericamente.

`compile_model_forward`

Compila el metodo `forward` del modelo:

```
def compile_model_forward(model, compile_forward):
    if compile_forward == 'off':
        return False
    if not hasattr(torch, 'compile'):
        return False
    kwargs = {'dynamic': True}
    if compile_forward != 'default':
        kwargs['mode'] = compile_forward
    model.forward = torch.compile(model.forward, **kwargs)
    return True
```

Razon tecnica:

- `ARCCompressor` no hereda de `torch.nn.Module`.
- Por tanto, compilar `model` completo no es la ruta natural.
- El punto menos invasivo es compilar el bound method `model.forward`.
- `dynamic=True` es necesario porque las formas dependen de cada puzzle ARC.

4.2 `train.py`

`take_step` ahora acepta:

```
def take_step(task, model, optimizer, train_step, train_history_logger,
              mixed_precision='off'):
```

El forward y el calculo de la reconstruccion se ejecutan dentro de:

```
with runtime_config.autocast_context(mixed_precision):
```

La parte importante es que los escalares MDL se fuerzan a FP32:

```
total_KL = torch.zeros(), device=logits.device, dtype=torch.float32)
for KL_amount in KL_amounts:
    total_KL = total_KL + torch.sum(KL_amount.float())

reconstruction_error = torch.zeros(), device=logits.device, dtype=torch.float32)
...
ce_sum = ce.float().sum(dim=(1, 2)).reshape(n_x_offsets, n_y_offsets)
...
reconstruction_error = reconstruction_error - logprob.float()

loss = total_KL.float() + 10 * reconstruction_error.float()
```

Motivo:

- `total_KL` mide la longitud de codigo del latente.
- `reconstruction_error` participa directamente en la perdida MDL.
- Si ambos se acumularan en BF16/FP16, podria degradarse la estabilidad numerica.
- El objetivo es acelerar operaciones tensoriales internas sin alterar la contabilidad de la perdida.

Tambien se anadio CLI al entrenamiento secuencial:

```
python train.py \
  --split training \
  --demo 1 \
  --n-iterations 2000 \
  --mixed-precision bf16 \
  --compile-forward off \
  --float32-matmul-precision high
```

Esto permite comprobar que el Eje D no depende exclusivamente del scheduler paralelo.

4.3 solve_task.py

La firma de solve_task ahora recibe las opciones del Eje D:

```
def solve_task(...,
                postprocess_stride=1,
                mixed_precision='off',
                compile_forward='off',
                float32_matmul_precision='high'):
```

Dentro del worker:

```
torch.set_default_device('cuda')
torch.cuda.set_device(gpu_id)
runtime_config.apply_torch_backend_settings(float32_matmul_precision)
```

Esto es necesario porque multiprocessing usa spawn: cada worker es un proceso Python independiente y debe configurar su propio backend.

Despues de crear el modelo:

```
model = arc_compressor.ARCCompressor(task)
runtime_config.compile_model_forward(model, compile_forward)
```

Y cada iteracion pasa el modo de precision:

```
train.take_step(task, model, optimizer, train_step, train_history_logger,
               mixed_precision=mixed_precision)
```

4.4 parallel_train.py

parallelize_runs ahora propaga a cada worker:

```
mixed_precision='off'
compile_forward='off'
float32_matmul_precision='high'
```

Los argumentos se pasan en worker_args:

```
worker_args = (
    task_names[i], split, 1e20, n_iterations,
    gpu_id, memory_dict, solutions_dict, error_queue,
    _loggers_dict, _progress_dict, postprocess_stride,
    mixed_precision, compile_forward, float32_matmul_precision,
)
```

Nuevos flags de CLI

```
--mixed-precision {off,bf16,fp16}
--compile-forward {off,default,reduce-overhead,max-autotune}
```

```
--compile-phase1
--float32-matmul-precision {highest,high,medium}
```

Uso recomendado inicial:

```
python parallel_train.py \
  --split training \
  --demo 20 \
  --max-workers 8 \
  --mixed-precision bf16 \
  --compile-forward off \
  --float32-matmul-precision high
```

Phase 1 y cache de VRAM

La cache de medicion de VRAM ahora incluye las opciones de runtime:

```
def _gpu_fingerprint(n_gpus, runtime_options=None):
    return {
        'n_gpus': n_gpus,
        'gpu_names': [...],
        'gpu_vram_total_gb': [...],
        'torch_version': torch.__version__,
        'runtime_options': runtime_options or {},
    }
```

Esto evita mezclar mediciones incompatibles. Por ejemplo, una medicion FP32 no deberia reutilizarse automaticamente para BF16 si el footprint cambia.

El runtime usado por Phase 1 se calcula asi:

```
memory_runtime_options = runtime_config.runtime_options_dict(
    mixed_precision=mixed_precision,
    compile_forward=compile_forward if compile_phase1 else 'off',
    float32_matmul_precision=float32_matmul_precision,
)
```

Decision importante:

- Phase 1 solo corre 2 iteraciones por task.
- `torch.compile` tiene warmup caro.
- Por defecto, Phase 1 no compila aunque Phase 2 compile.
- Solo se compila Phase 1 con `--compile-phase1`.

4.5 `profile_parallel_train.py`

El profiler se extendio con metricas especificas para comparar el Eje D.

Extrae los flags pasados a `parallel_train.py`:

```
def _extract_eje_d_config(passthrough):
    return {
        'mixed_precision': _passthrough_value(...),
        'compile_forward': _passthrough_value(...),
        'compile_phase1': '--compile-phase1' in passthrough,
        'float32_matmul_precision': _passthrough_value(...),
    }
```

Guarda en el resumen JSON:

```
eje_d_metrics = {
    'config': _extract_eje_d_config(passthrough),
    'tree_cpu_mean_pct': ...,
    'tree_cpu_per_worker_pct': ...,
    'gpu_util_per_worker_pct': ...,
    'gpu_cpu_balance': ...,
}
```

```
}  
    'vram_per_max_worker_mb': ...,
```

Interpretacion de las nuevas metricas:

Metrica	Significado
config	Flags exactos usados en la run.
tree_cpu_mean_pct	CPU agregada del proceso padre + workers.
tree_cpu_per_worker_pct	Coste CPU medio por worker activo.
gpu_util_per_worker_pct	Utilizacion GPU media dividida por concurrencia media.
gpu_cpu_balance	Relacion entre utilizacion GPU y CPU del sistema. Mayor suele ser mejor.
vram_per_max_worker_mb	VRAM maxima observada dividida por maximo de workers vivos.

El comando `--compare` ahora muestra tambien la seccion:

```
Eje D hardware balance
```

Con esto se puede distinguir si una optimizacion:

- mejora wall-clock;
- solo sube uso de GPU pero empeora throughput;
- reduce CPU por worker;
- aumenta VRAM y por tanto reduce la concurrencia posible.

5. Validaciones realizadas en la sesion

5.1 Validacion de sintaxis

Comando:

```
python -m py_compile runtime_config.py train.py solve_task.py parallel_train.py profile_parallel_train.py
```

Resultado final:

```
sin errores de sintaxis
```

Durante la implementacion aparecieron dos errores de indentacion, ambos corregidos:

1. `parallel_train.py: IndentationError` en `load_memory_cache`.
2. `profile_parallel_train.py: IndentationError` en el bloque de comparacion del Eje D.

Tras corregirlos, `py_compile` paso correctamente.

5.2 Validacion de CLI

Comandos:

```
python parallel_train.py --help  
python profile_parallel_train.py --help
```

Resultado:

- `parallel_train.py` mostro los nuevos flags del Eje D.
- `profile_parallel_train.py` cargo correctamente.

5.3 Validacion estatica del editor

Se revisaron estos archivos con las herramientas del editor:

```
runtime_config.py  
train.py
```



```
solve_task.py
parallel_train.py
profile_parallel_train.py
```

Resultado:

```
No errors found
```

5.4 Smoke test FP32/BF16

Comando ejecutado:

```
source arcagi/bin/activate && python - <<'PY'
import torch
import preprocessing
import arc_compressor
import solution_selection
import train
import runtime_config

runtime_config.apply_torch_backend_settings('high')
print('cuda_available', torch.cuda.is_available())
task = preprocessing.preprocess_tasks('training', ['272f95fa'])[0]
for precision in ('off', 'bf16'):
    model = arc_compressor.ARCCompressor(task)
    optimizer = torch.optim.Adam(model.weights_list, lr=0.01, betas=(0.5, 0.9))
    logger = solution_selection.Logger(task, postprocess_stride=999)
    train.take_step(task, model, optimizer, 0, logger, mixed_precision=precision)
    print(precision, len(logger.loss_curve), type(logger.loss_curve[-1]).__name__)

PY
```

Salida relevante:

```
cuda_available True
off 1 Tensor
bf16 1 Tensor
```

Interpretacion:

- CUDA/ROCm estaba disponible para PyTorch.
- `take_step` funciona en FP32 (`off`).
- `take_step` funciona en BF16.
- El logger recibio una entrada de loss en ambos casos.

5.5 Prueba de `torch.compile`

Comando ejecutado:

```
source arcagi/bin/activate && python - <<'PY'
import torch
import preprocessing
import arc_compressor
import solution_selection
import train
import runtime_config

runtime_config.apply_torch_backend_settings('high')
task = preprocessing.preprocess_tasks('training', ['272f95fa'])[0]
model = arc_compressor.ARCCompressor(task)
compiled = runtime_config.compile_model_forward(model, 'reduce-overhead')
optimizer = torch.optim.Adam(model.weights_list, lr=0.01, betas=(0.5, 0.9))
logger = solution_selection.Logger(task, postprocess_stride=999)
train.take_step(task, model, optimizer, 0, logger, mixed_precision='off')
print('compiled', compiled, 'loss_entries', len(logger.loss_curve))

PY
```

Resultado observado:

- la ejecucion supero el timeout de 120 s;
- no devolvio salida util antes del timeout;
- se comprobo de nuevo la salida del terminal;
- seguia sin finalizar;
- se mato el terminal para no dejar una compilacion colgada.

Conclusion:

torch.compile queda implementado, pero experimental.
No debe activarse por defecto hasta demostrar amortizacion en runs largos.

6. Comandos de uso

6.1 Entrenamiento secuencial baseline

```
python train.py \  
  --split training \  
  --demo 1 \  
  --n-iterations 2000 \  
  --mixed-precision off \  
  --compile-forward off \  
  --float32-matmul-precision highest
```

6.2 Entrenamiento secuencial BF16

```
python train.py \  
  --split training \  
  --demo 1 \  
  --n-iterations 2000 \  
  --mixed-precision bf16 \  
  --compile-forward off \  
  --float32-matmul-precision high
```

6.3 Entrenamiento paralelo baseline

```
python parallel_train.py \  
  --split training \  
  --demo 20 \  
  --max-workers 8 \  
  --mixed-precision off \  
  --compile-forward off \  
  --float32-matmul-precision highest
```

6.4 Entrenamiento paralelo BF16

```
python parallel_train.py \  
  --split training \  
  --demo 20 \  
  --max-workers 8 \  
  --mixed-precision bf16 \  
  --compile-forward off \  
  --float32-matmul-precision high
```

6.5 Entrenamiento paralelo con torch.compile experimental

```
python parallel_train.py \  
  --split training \  
  --demo 16 \  
  --max-workers 2 \  
  --mixed-precision bf16 \  
  --compile-forward reduce-overhead \  
  --float32-matmul-precision high
```


No activar inicialmente:

```
--compile-phase1
```

Motivo: Phase 1 solo ejecuta 2 iteraciones por tarea y casi nunca amortiza la compilacion.

7. Benchmarking antes/despues

7.1 Comparacion BF16 vs baseline

Baseline:

```
python profile_parallel_train.py --label d_baseline --gpu-mode sysfs -- \
  --split training --demo 20 --max-workers 8 \
  --mixed-precision off --compile-forward off \
  --float32-matmul-precision highest
```

BF16:

```
python profile_parallel_train.py --label d_bf16 --gpu-mode sysfs -- \
  --split training --demo 20 --max-workers 8 \
  --mixed-precision bf16 --compile-forward off \
  --float32-matmul-precision high
```

Comparacion:

```
python profile_parallel_train.py --compare \
  .profile/d_baseline_summary.json \
  .profile/d_bf16_summary.json
```

7.2 Prueba profunda de torch.compile

La prueba profunda debe medir amortizacion en paralelo, no solo si compila.

Nivel 1: una tarea, un worker

```
python profile_parallel_train.py --label compile_off_1w --gpu-mode sysfs -- \
  --split training --demo 1 --max-workers 1 \
  --mixed-precision off \
  --compile-forward off \
  --float32-matmul-precision high
```

```
python profile_parallel_train.py --label compile_on_1w --gpu-mode sysfs -- \
  --split training --demo 1 --max-workers 1 \
  --mixed-precision off \
  --compile-forward reduce-overhead \
  --float32-matmul-precision high
```

Objetivo: comprobar si el coste de compilacion ya destruye la run mas pequena.

Nivel 2: varias tareas, un worker

```
python profile_parallel_train.py --label compile_off_4tasks_1w --gpu-mode sysfs -- \
  --split training --demo 4 --max-workers 1 \
  --mixed-precision off \
  --compile-forward off \
  --float32-matmul-precision high
```

```
python profile_parallel_train.py --label compile_on_4tasks_1w --gpu-mode sysfs -- \
  --split training --demo 4 --max-workers 1 \
```

```
--mixed-precision off \  
--compile-forward reduce-overhead \  
--float32-matmul-precision high
```

Objetivo: comprobar si PyTorch/Inductor reutiliza algo entre tareas o si cada tarea paga el coste completo.

Nivel 3: concurrencia escalonada

```
for W in 2 4 8; do  
  python profile_parallel_train.py --label compile_off_w${W} --gpu-mode sysfs -- \  
    --split training --demo 16 --max-workers ${W} \  
    --mixed-precision off \  
    --compile-forward off \  
    --float32-matmul-precision high  
  
  python profile_parallel_train.py --label compile_on_w${W} --gpu-mode sysfs -- \  
    --split training --demo 16 --max-workers ${W} \  
    --mixed-precision off \  
    --compile-forward reduce-overhead \  
    --float32-matmul-precision high  
  
  python profile_parallel_train.py --compare \  
    .profile/compile_off_w${W}_summary.json \  
    .profile/compile_on_w${W}_summary.json  
done
```

Interpretacion:

- Si `W=2` mejora pero `W=8` empeora, hay tormenta de compilacion CPU/RAM.
- Si todos empeoran, `torch.compile` no compensa con el punto actual.
- Si todos mejoran y `gpu_cpu_balance` sube, la compilacion podria compensar.
- Si sube `gpu_util_mean_pct` pero baja `workers_per_hour`, la GPU se usa mejor pero el throughput global empeora.

Nivel 4: mejor candidato real, BF16 + compile

```
python profile_parallel_train.py --label bf16_compile_off_w4 --gpu-mode sysfs -- \  
  --split training --demo 16 --max-workers 4 \  
  --mixed-precision bf16 \  
  --compile-forward off \  
  --float32-matmul-precision high
```

```
python profile_parallel_train.py --label bf16_compile_on_w4 --gpu-mode sysfs -- \  
  --split training --demo 16 --max-workers 4 \  
  --mixed-precision bf16 \  
  --compile-forward reduce-overhead \  
  --float32-matmul-precision high
```

Esta es la prueba mas importante en la practica: no pregunta si compile mejora FP32, sino si mejora sobre el candidato realista BF16.

7.3 Criterios de aceptacion para `torch.compile`

`torch.compile` solo deberia activarse por defecto si cumple, como minimo:

```
workers_per_hour mejora >= 10 %  
mean_worker_lifetime_s baja  
cpu_saturation_fraction no sube de forma fuerte  
gpu_cpu_balance sube  
vram_per_max_worker_mb no aumenta lo bastante como para reducir concurrencia
```

Si mejora menos de 5 %, no merece activarse por defecto. El riesgo operativo no compensa el beneficio marginal.

8. Limitaciones actuales

8.1 `torch.compile` puede no amortizar

La prueba realizada mostro que una unica iteracion compilada no termino antes de 120 s. Esto no prueba que compile sea inutil, pero si muestra que el warmup es caro.

En `parallel_train.py`, el problema es mas serio que en un entrenamiento monolitico:

- cada tarea vive en su propio proceso;
- cada proceso crea un modelo nuevo;
- cada modelo puede tener shapes distintas;
- el cache de compilacion puede no reutilizarse lo suficiente;
- muchos workers compilando a la vez pueden saturar CPU/RAM.

8.2 No se midio todavia el tiempo interno setup/train

El profiler mide bien el efecto externo:

- wall-clock;
- workers/h;
- vida media de worker;
- CPU;
- GPU;
- VRAM.

Pero aun no separa dentro de cada worker:

```
setup_time_s      = carga de task + creacion modelo + compile
train_loop_time_s = tiempo solo de iteraciones
iters_per_second  = n_train_iterations / train_loop_time_s
```

Esto seria una mejora futura util para diagnosticar `torch.compile` con mas precision.

8.3 BF16 debe validarse por accuracy

La smoke test prueba que BF16 ejecuta, no que preserve la misma distribucion de soluciones. Para aceptar BF16 en runs grandes hay que comparar:

- `workers_per_hour`;
- `n_solved` en training/evaluation cuando haya ground truth;
- estabilidad de curvas de KL;
- errores NaN/Inf;
- diferencia en `submission_{split}.json`.

9. Recomendacion operativa

Orden recomendado de pruebas:

1. Baseline FP32 sin compile.
2. BF16 sin compile.
3. BF16 con distintos `--max-workers`.
4. `torch.compile` con `--max-workers 1` o `2`.
5. `torch.compile` con concurrencia escalonada.
6. BF16 + `torch.compile` solo si el punto anterior mejora throughput.

Configuracion candidata inicial:

```
python parallel_train.py \
  --split training \
  --demo 20 \
  --max-workers 8 \
  --mixed-precision bf16 \
  --compile-forward off \
  --float32-matmul-precision high
```

Configuracion experimental de compile:

```
python parallel_train.py \
  --split training \
  --demo 16 \
  --max-workers 2 \
  --mixed-precision bf16 \
```

```
--compile-forward reduce-overhead \  
--float32-matmul-precision high
```

No usar de entrada:

```
--compile-phase1
```

10. Resumen final de la sesion

Se implemento el Eje D como una capa de ejecucion configurable y medible.

Archivos creados:

```
runtime_config.py
```

Archivos modificados:

```
train.py  
solve_task.py  
parallel_train.py  
profile_parallel_train.py  
Docs/Architecture/Axi_D.md
```

Cambios funcionales:

- precision mixta configurable (`off`, `bf16`, `fp16`);
- `torch.set_float32_matmul_precision` configurable;
- `torch.compile` opcional sobre `ARCCompressor.forward`;
- propagacion de opciones desde CLI hasta cada worker paralelo;
- cache de VRAM sensible a opciones de runtime;
- metricas nuevas en profiler para comparar Eje D;
- comandos de benchmark antes/despues;
- documentacion de limitaciones y protocolo de prueba profunda.

Estado de validacion:

```
py_compile: OK  
VS Code static errors: OK  
parallel_train.py --help: OK  
profile_parallel_train.py --help: OK  
FP32 one-step smoke test: OK  
BF16 one-step smoke test: OK  
torch.compile one-step test: timeout > 120 s, terminal killed
```

Decision tecnica final:

```
BF16 + matmul_precision='high' es la mejora practica inicial.  
torch.compile queda disponible como experimento controlado, no como default.
```